Internal Automation Project Plan

Automating Web Article Retrieval, Summarization, and Archival with n8n

1. Introduction: Purpose and Expected Benefits

This project builds an n8n workflow that takes a URL, retrieves the page content, generates an AI summary, and archives the result to a shared Google Drive folder — with no manual copy-pasting. Expected benefits: minutes of manual reading/writing time saved per article, a consistently formatted archive the whole team can search, and a reusable pattern (fetch → process → store) that extends to other internal processes.

2. Selected Internal Process and Rationale

Selected process: retrieval, AI summarization, and archival of external reference articles (e.g., vendor advisories or technical documentation) for team knowledge-sharing.

- **Repetitive:** High frequency, low complexity — the same three steps (fetch, summarize, save) repeat for every article, making it a clean fit for a linear automation.
- **Time-intensive:** Reading and condensing a technical article by hand is time consuming; an LLM can do the first-pass summary in seconds.
- **API-accessible:** Every step in this process (page fetch, LLM call, file storage) is exposed as an HTTP-based API, so the whole chain can be automated without custom backend code.

3. Workflow Walkthrough in n8n

Step 1 — Trigger

The workflow starts from the “When clicking ‘Execute workflow’” node. For this first version, a manual trigger keeps testing fast; production use would swap this for a Webhook node so the workflow can be called directly from Slack or a form.

Step 2 — Retrieve the source content

An HTTP Request node sends a GET call to the Jina AI Reader endpoint (r.jina.ai/<target URL>). Jina's Reader API strips ads, navigation, and markup, returning clean text — removing the need to write and maintain a custom HTML parser.

Step 3 — Summarize with an LLM

The cleaned text passes into a LangChain “Summarization Chain” node, which is wired to an OpenRouter Chat Model node as its language model. OpenRouter provides a single API that can route to different underlying models, so the model choice can change later without rebuilding the chain.

Step 4 — Package the result

The “Convert to File” node turns the chain's text output into a proper text file (Convert to Text File), which downstream nodes can treat as a binary attachment.

Step 5 — Archive to Google Drive

Two Google Drive nodes close the loop: “Upload file” writes the summary into Drive, and “Create folder” organizes archives (e.g., by date or topic) so the knowledge base stays browsable rather than becoming one flat folder of files.

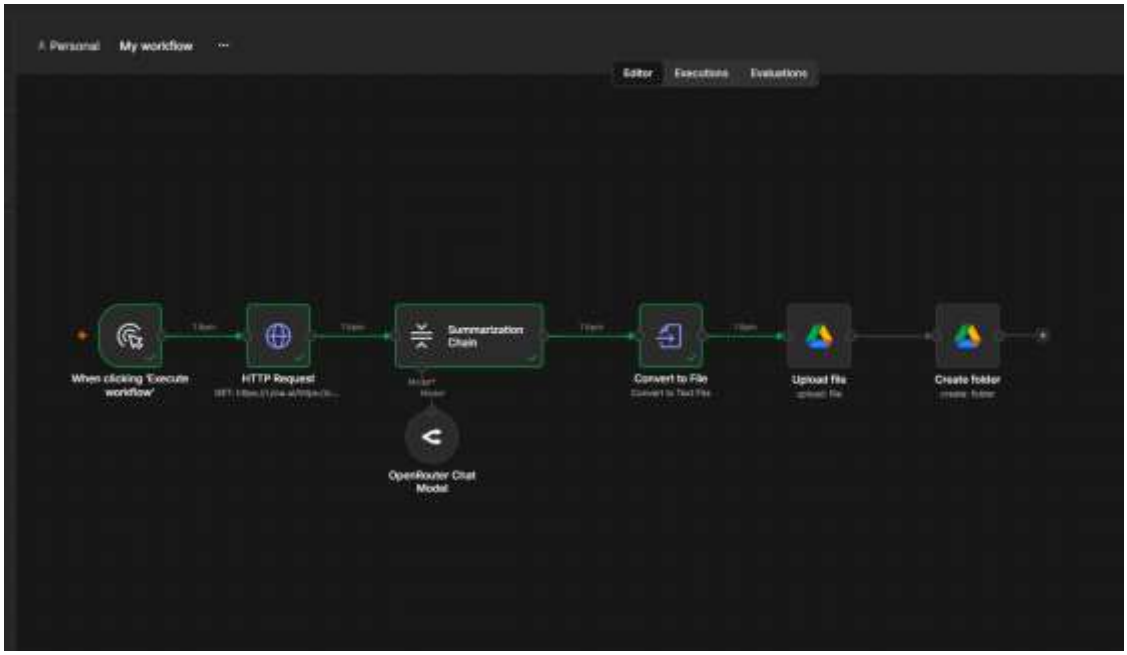


Figure 1. Completed n8n workflow — trigger through summarization to Google Drive archival.

4. Integration Points: APIs and Webhooks

Every node in this workflow is, functionally, an API call. The table below maps each integration point to the service behind it.

Integration Point	Mechanism	Purpose in This Workflow
Manual trigger	n8n “Execute Workflow” trigger node	Starts the run on demand; swappable later for a Webhook or Schedule trigger with no change to downstream logic.
Source content retrieval	HTTP Request node calling the Jina AI Reader API (r.jina.ai)	Converts a live webpage into clean, LLM-ready text via a single authenticated GET request — no custom scraper to build or maintain.
Summarization	LangChain Summarization Chain node + OpenRouter Chat Model API	Sends the retrieved text to a hosted LLM through OpenRouter’s API and returns a concise summary.
File packaging	Convert to File node (Convert to Text File)	Transforms the summary output from JSON into a plain-text binary file n8n can hand off to storage.
Archival	Google Drive API — Upload File and Create Folder nodes	Authenticates via OAuth2 and writes the summary into a dated folder in a shared Drive, closing the loop from source to archive.

5. Conclusion: Logic and API Usage

The workflow follows a simple, linear logic — fetch → summarize → package → store — with each stage handled by a dedicated API rather than custom code. This is the core efficiency argument for low-code automation: n8n's role is to pass data between existing APIs (Jina AI Reader, OpenRouter, Google Drive) in the right order and shape, rather than reimplementing what those services already do well.

The one architectural decision worth calling out is the choice of a manual trigger over a Webhook for this version. It was a deliberate simplification to validate the fetch-summarize-store logic before adding a triggering mechanism; converting the manual trigger to a Webhook node is a same-day change and does not require touching any node downstream of it — evidence that the workflow's modular, API-driven structure is what makes it easy to extend.